

Commercial API Hub & Token-Based Access System
Technical Documentation
Piezo Pico-to-Femtotechnology Sensors Inc.
Version 1.0 — August 2026

1. Executive Summary

Piezo-Pico to Femtotechnology Sensors Inc. provides a unified commercial API hub that manages access to:

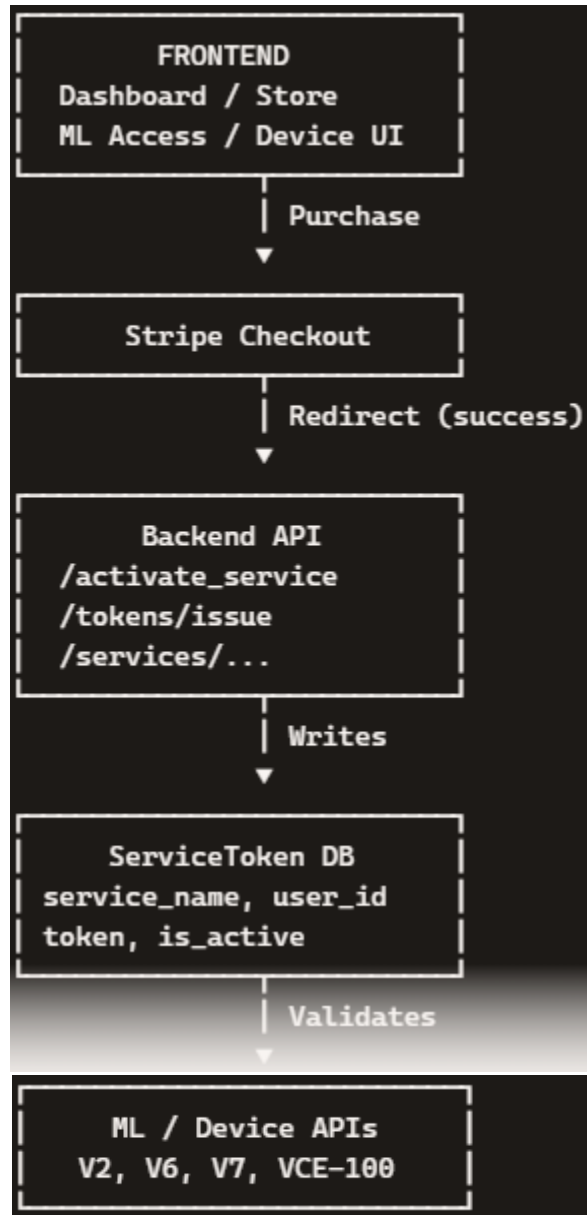
- ML Models (V2, V6, V7, VCE-100, Multiclassify)
- Device APIs
- Consulting services
- Course modules
- Marketplace products

All services use a **token-based activation system** powered by:

- Stripe Checkout
- Backend activation logic
- ServiceToken database
- Token issuance endpoints
- API gateways validating tokens

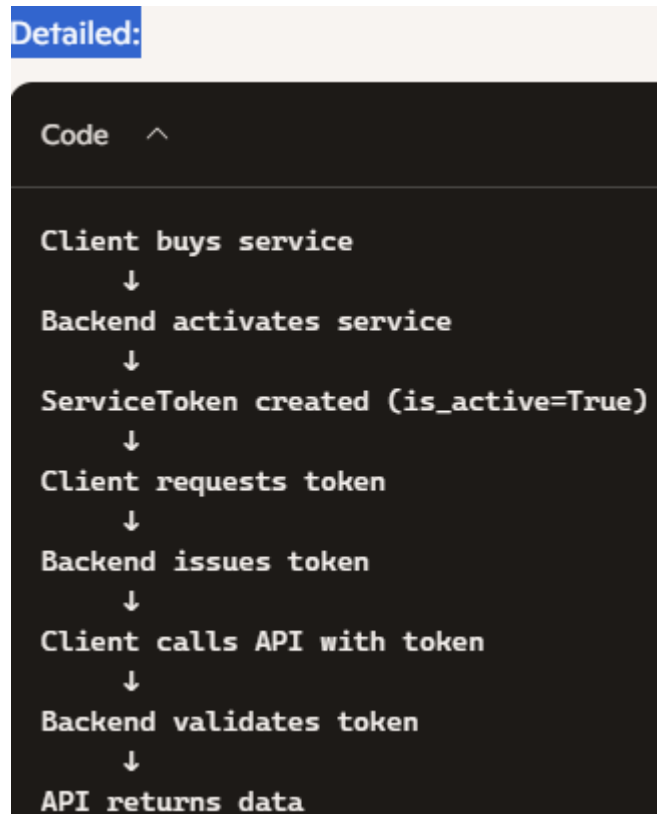
This document explains how a client purchases access, how tokens are issued, and how the backend validates service access.

2. High-Level Architecture Diagram



3. Token Lifecycle Diagram

Purchase → Activation → Token Issued → API Access → Validation → Response



4. Unified Purchase → Activation → Token → API Access Flow

This applies to **ALL services**:

- ML Models
- Device APIs
- Consulting
- Courses
- Marketplace digital products

```
1. Client selects service in Store
2. Frontend sends purchase request → Stripe Checkout
3. Stripe processes payment
4. Stripe redirects back with payment=success
5. Dashboard detects success
6. Dashboard calls backend /activate_service
7. Backend creates ServiceToken entry:
    user_id
    service_name
    is_active=True
8. Client opens service dashboard
9. Frontend checks if token exists
10. If missing → frontend requests /tokens/issue
11. Backend issues token + stores it
12. Frontend calls API with token
13. Backend validates token
14. API returns service data
```

5. ML Model V2 Access

How a client gets ML V2 access

1. The client opens the Store and selects **ML Model V2**.
2. The frontend sends the purchase request to **Stripe Checkout**.
3. Stripe processes the payment and redirects back with payment=success.
4. The dashboard detects the success and calls the backend to activate ML V2.
5. The backend creates a **ServiceToken** entry:
 - service_name="v2"
 - is_active=True
6. When the client clicks **ML Access**, the frontend checks if a valid token exists.
7. If no token exists, the frontend requests /tokens/issue?service=v2.
8. The backend issues a token and stores it in the database.
9. The frontend calls /services/ml/v2?token=XYZ.
10. The backend validates the token and returns ML V2 model output.

6. Generalized Flow for ALL Services

ML Models

- v2
- v6
- v7
- vce100_v2
- vce100_v6
- vce100_compare
- multiclass_v6

All follow the same activation + token + API validation flow.

Device APIs

- Device purchase → activation → token → /device/api?token=XYZ

Consulting

- Consulting package → activation → token → /consulting/api?token=XYZ

Courses

- Course access → activation → token → /courses/api?token=XYZ

Marketplace Digital Products

- Product → activation → token → /products/api?token=XYZ

7. Company Hub Architecture

The dashboard is now a **full company hub**, containing:

- Store
- Cart
- Checkout
- ML Models
- Device APIs
- Consulting
- Courses

- Government portal
- Subscription system
- Token access system
- Documentation viewer
- Marketplace navigation bar

This is a **commercial API platform**, not just a dashboard.

8. Business Components API Model

Each service becomes a business component:

ML_V2

ML_V6

ML_V7

VCE100_V2

VCE100_V6

VCE100_COMPARE

MULTICLASS

DEVICE_API

CONSULTING_API

COURSE_API

SUBSCRIPTION_API

Each component exposes:

/activate_service

/tokens/issue

/services/<component>?token=XYZ

This is a scalable architecture for selling:

- ML models
- Data pipelines
- Device integrations

- Consulting automation
- Course modules
- Government APIs
- Marketplace digital products

9. Security Model

Token Validation

- Tokens are tied to user_id
- Tokens expire
- Tokens are hashed
- Tokens are validated per request

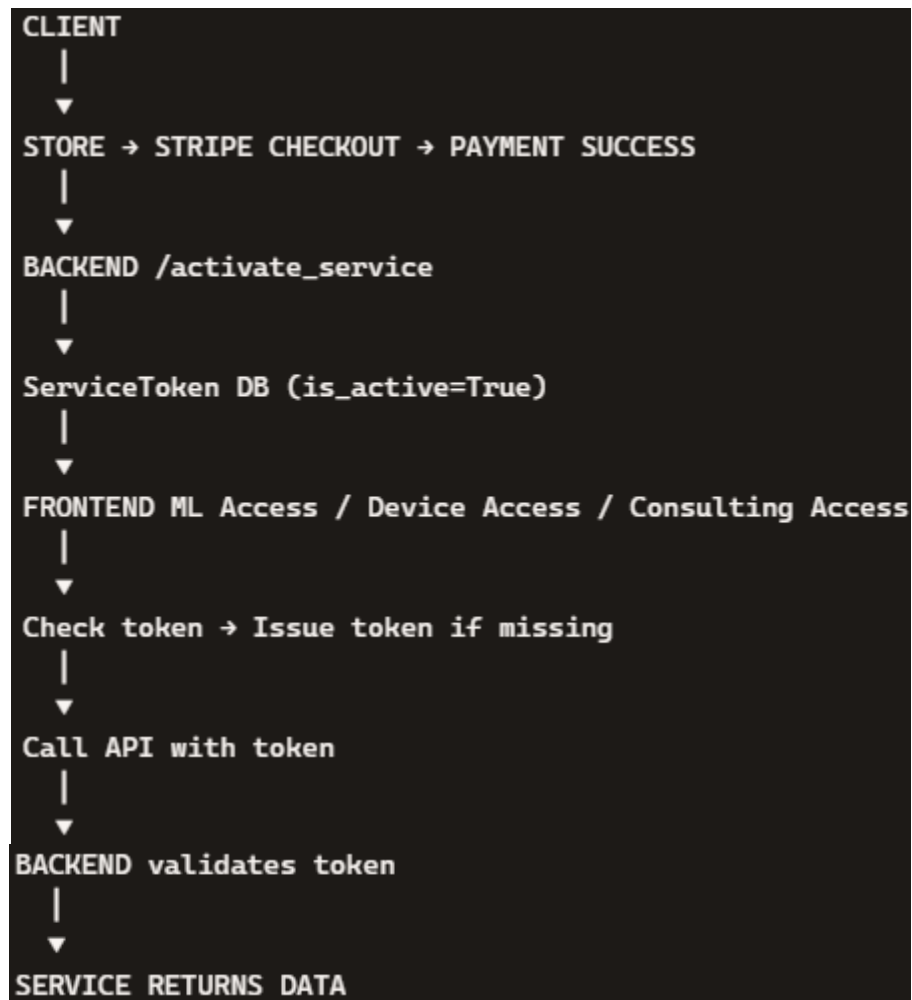
ServiceToken Table

```
id
user_id
service_name
token
is_active
created_at
expires_at
```

API Gateway Rules

- Reject missing tokens
- Reject expired tokens
- Reject mismatched service_name
- Reject invalid user_id
- Log all access

11. Final Consolidated Diagram



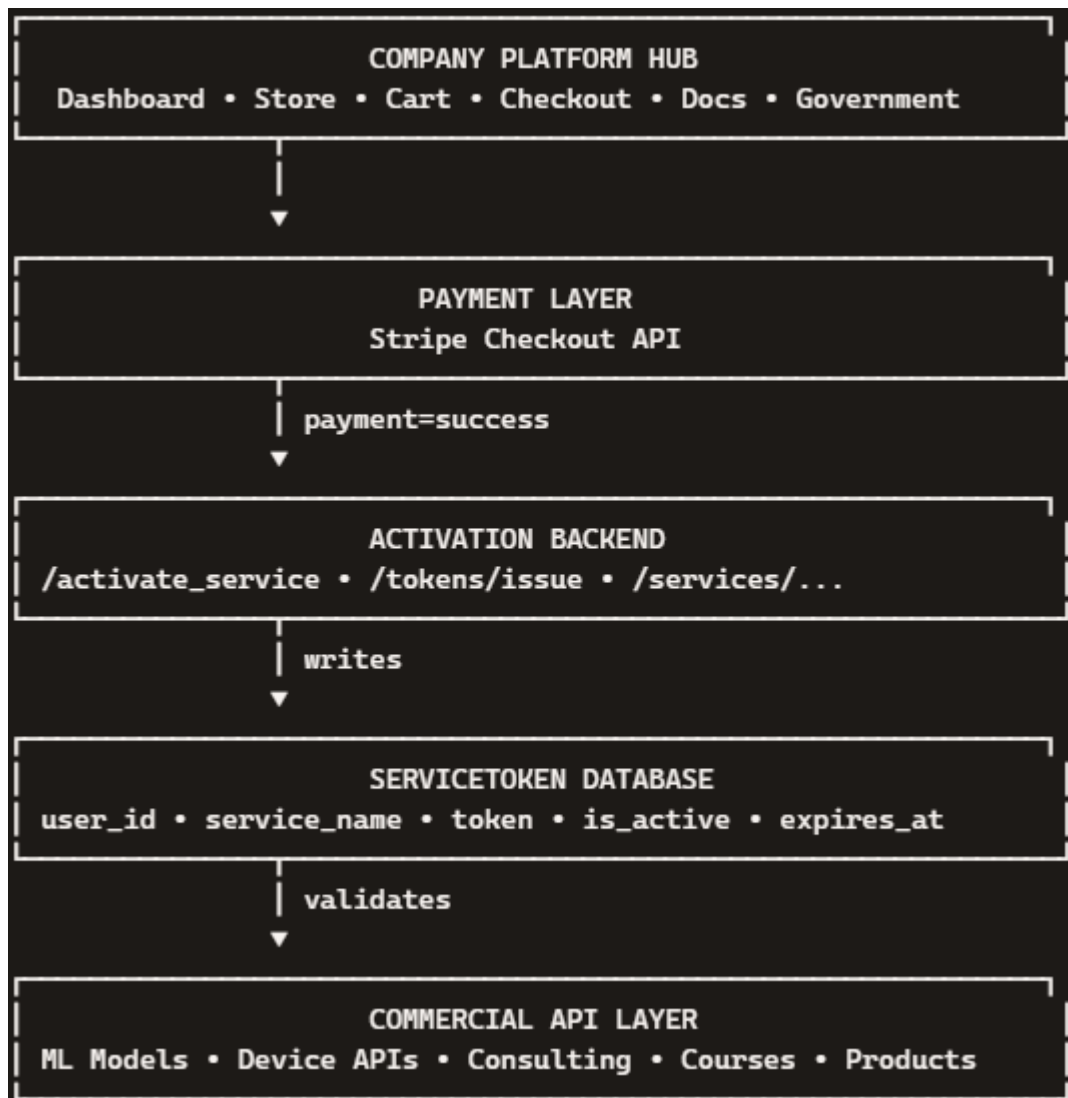
Top-Level System Architecture

The company platform is built as a modular, token-secured, API-driven ecosystem. Every product — ML models, device APIs, consulting packages, courses, and marketplace digital items — follows the same lifecycle:

Purchase → Activation → Token → API Access → Validation → Response

This creates a scalable, secure, and unified commercial architecture.

11.1 Top-Level System Architecture



11.2 Platform Components

Frontend Layer

- Dashboard (React / Next.js)
- Store, Cart, Checkout
- ML Access, Device Access, Consulting Access
- Token-aware UI components
- Documentation viewer
- Government portal
- Subscription status

Backend Layer

- FastAPI / Node API gateway
- Stripe payment webhooks
- Token issuance endpoints
- Service activation logic
- ML model inference endpoints
- Device API endpoints
- Consulting automation endpoints
- Course content delivery endpoints

Database Layer

- ServiceToken table
- Users table
- Purchases table
- Logs table
- APIUsage table

Commercial API Layer

- /services/ml/v2
- /services/ml/v6

- /services/ml/v7
- /services/ml/vce100_v2
- /services/ml/vce100_v6
- /services/ml/vce100_compare
- /services/ml/multiclass
- /device/api
- /consulting/api
- /courses/api
- /products/api

11.3 Unified Token-Based Access Model

Every service in the company uses the same token model:

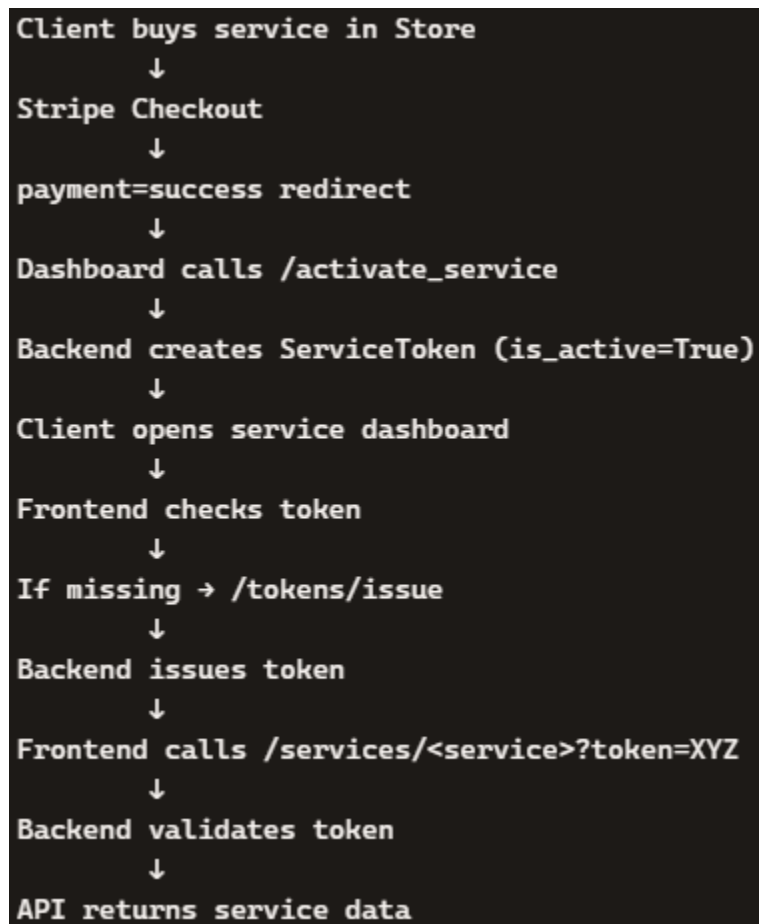
```
ServiceToken {  
  id  
  user_id  
  service_name  
  token  
  is_active  
  created_at  
  expires_at  
}
```

Tokens are:

- Issued per service
- Validated per request
- Bound to user_id
- Expiring
- Logged
- Required for all API calls

This creates a **consistent security model** across all business units.

11.4 Company-Wide Purchase → Activation → Access Flow



11.5 Company Hub Vision

Your platform is now a commercial API hub capable of selling:

- ML inference models
- Device sensor APIs
- Consulting automation
- Course modules
- Government-level biosensing APIs
- Marketplace digital products

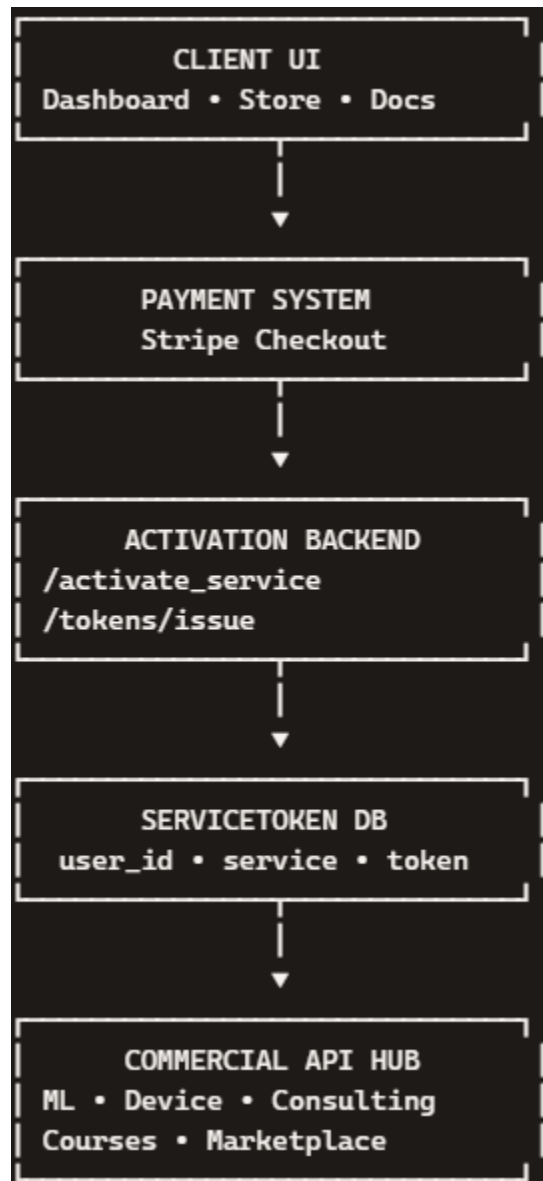
All powered by:

- Stripe
- Token system

- Unified activation logic
- Modular API endpoints
- Scalable architecture

This is the foundation for a full enterprise ecosystem.

11.6 Company-Level Architecture Diagram (Consolidated)



12. Per-Service Architecture Diagrams

Unified Token-Based Access for All Commercial Services

Each service in your ecosystem follows the same lifecycle:

Purchase → Activation → Token Issuance → API Access → Validation → Respon

But each service has its own service_name, API endpoint, and business logic.

Below are the per-service diagrams.

12.1 ML Models (V2, V6, V7, VCE-100, Multiclassify)

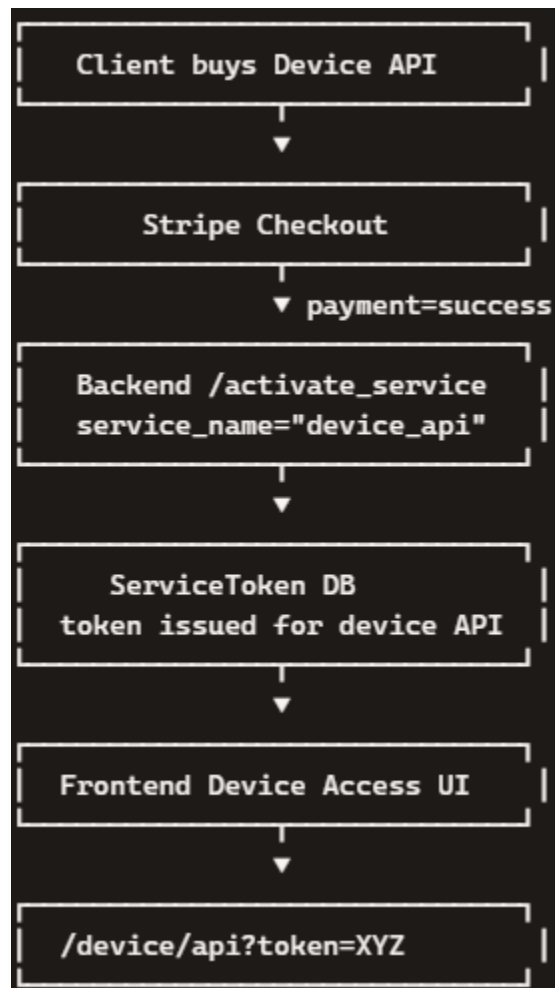


ML Model Endpoints

- /services/ml/v2
- /services/ml/v6
- /services/ml/v7

- /services/ml/vce100_v2
- /services/ml/vce100_v6
- /services/ml/vce100_compare
- /services/ml/multiclass

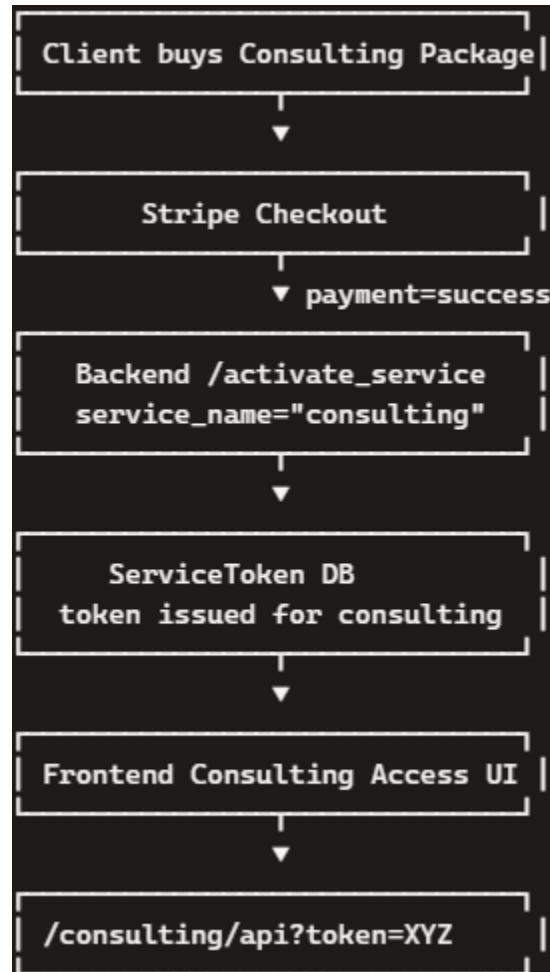
12.2 Device API Access



Device Endpoints

- /device/api
- /device/sensors
- /device/stream
- /device/diagnostics

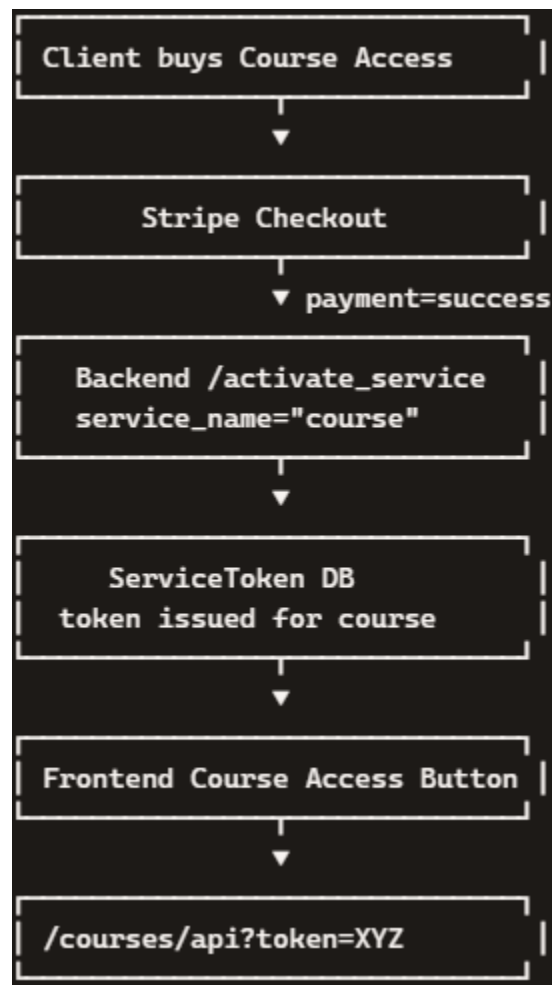
12.3 Consulting Services



Consulting Endpoints

- /consulting/api
- /consulting/schedule
- /consulting/history
- /consulting/analysis

12.4 Course Access



Course Endpoints

- /courses/api
- /courses/modules
- /courses/content
- /courses/progress

12.5 Marketplace Digital Products



Product Endpoints

- `/products/api`
- `/products/download`
- `/products/license`

12.6 Summary Table (All Services)

Service	API Endpoint
ML V2	/services/ml/v2
ML V6	/services/ml/v6
ML V7	/services/ml/v7
VCE-100 V2	/services/ml/vce100_v2
VCE-100 V6	/services/ml/vce100_v6
VCE-100 Compare	/services/ml/vce100_compare
Multiclassify	/services/ml/multiclass
Device API	/device/api
Consulting	/consulting/api
Courses	/courses/api
Marketplace Products	/products/api

13 Database Schema Diagrams

This section shows how **every service** — ML models, devices, consulting, courses, marketplace — is supported by a clean, scalable database schema.

- 13.1 Core Database Schema
- 13.2 ServiceToken Table Diagram
- 13.3 Purchases Table Diagram
- 13.4 Users Table Diagram
- 13.5 APIUsage Logging Diagram
- 13.6 Relationships Diagram

Unified Token-Based Access Architecture

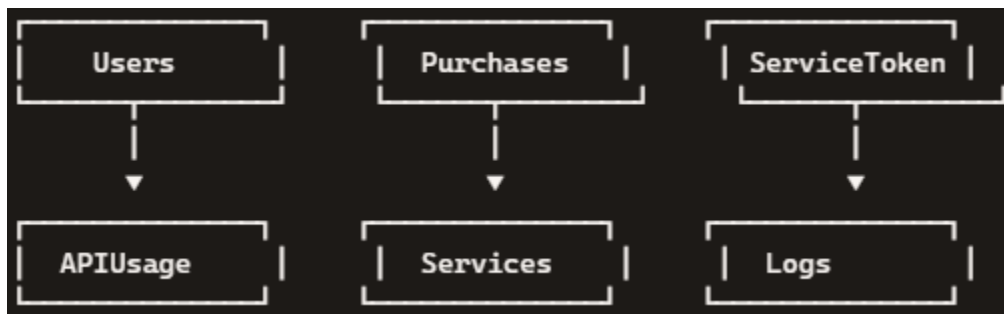
The platform uses a modular database design that supports:

- Stripe payments
- Service activation
- Token issuance
- API validation
- Usage logging
- Multi-service access

- Marketplace products
- ML model access
- Device API access
- Consulting access
- Course access

Below are the core schema diagrams.

13.1 Core Database Schema Overview



This structure supports **all commercial services**.

13.2 ServiceToken Table Diagram

The heart of company's commercial API system

ServiceToken
id (PK)
user_id (FK → Users.id)
service_name (string)
token (string, hashed)
is_active (boolean)
created_at (datetime)
expires_at (datetime)

Purpose

- Stores activation state
- Stores issued tokens
- Validates access
- Supports all services (ML, device, consulting, courses, products)

13.3 Purchases Table Diagram

Tracks Stripe transactions

Purpose

- Records every Stripe Checkout purchase
- Links purchases to service activation
- Supports refunds, audits, and compliance

13.4 Users Table Diagram

Core identity table

Users
id (PK)
email (string)
password_hash (string)
role (string: client, admin, gov)
created_at (datetime)
updated_at (datetime)

Purpose

- Stores user identity
- Supports roles (client, admin, government)
- Links to ServiceToken, Purchases, APIUsage

13.5 APIUsage Table Diagram

Tracks usage of ML models, devices, consulting, courses

APIUsage
id (PK) user_id (FK → Users.id) service_name (string) endpoint (string) timestamp (datetime) token_used (string, hashed) success (boolean) response_time_ms (integer)

Purpose

- Logs every API call
- Supports analytics
- Supports billing
- Supports fraud detection
- Supports government compliance

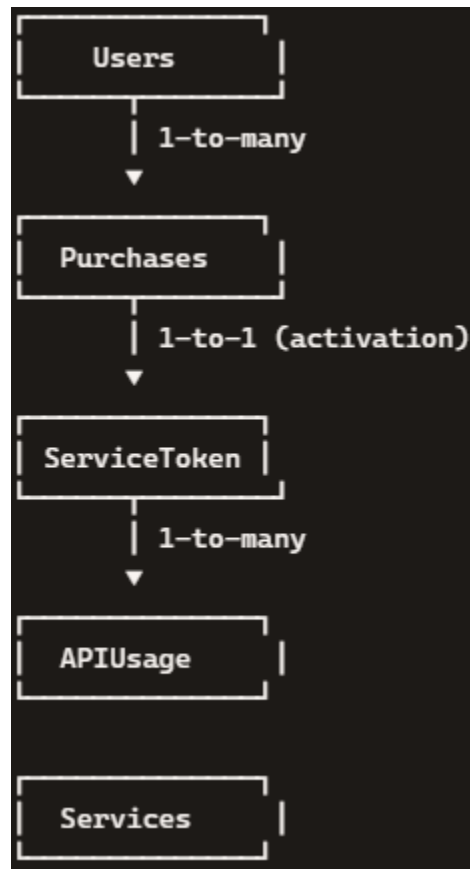
13.6 Services Table Diagram

Services
id (PK) service_name (string) description (string) price (decimal) category (ml, device, consulting, course) created_at (datetime)

Purpose

- Central registry of all services
- Used by Store, Checkout, Activation

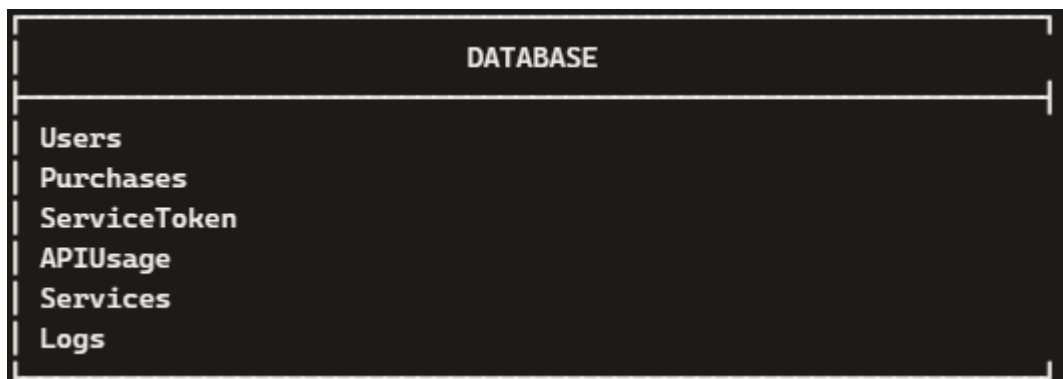
13.7 Relationships Diagram (Full Company Architecture)



Explanation

- **Users** buy services → **Purchases**
- Purchases activate → **ServiceToken**
- Tokens allow access → **APIUsage** logs calls
- **Services** define all available commercial APIs

3.8 Full Company Database Architecture Diagram



Relationships:

Users → Purchases → ServiceToken → APIUsage

Services → Purchases → ServiceToken

Summary:

Unified Commercial API Architecture

© 2026 Piezo-Pico to Femtotechnology Sensors Inc.

Enterprise Architecture Summary (Sections 1–13)

Piezo-Pico to Femtotechnology Sensors Inc. has established a unified, token-secured commercial API ecosystem that integrates machine learning models, device APIs, consulting services, course modules, and marketplace digital products into a single scalable company hub. The platform is built on a consistent lifecycle — **Purchase** → **Activation** → **Token** → **API Access** → **Validation** → **Response** — ensuring that every service follows the same secure and modular architecture.

Section 1 introduced the purpose of the system: a unified commercial API hub capable of selling ML inference models, device sensor APIs, consulting automation, course modules, and digital products.

Section 2 outlined the high-level architecture, showing how the frontend, Stripe Checkout, backend activation logic, ServiceToken database, and commercial API layer interact.

Section 3 detailed the token lifecycle, describing how tokens are issued, stored, validated, and used for API access.

Section 4 generalized the purchase-to-access flow for all services, establishing the core pattern used across the platform.

Section 5 provided the 10-step ML V2 access flow, demonstrating how Stripe, activation, token issuance, and ML model APIs work together.

Section 6 expanded this flow to all services — ML models, devices, consulting, courses, and marketplace products — showing the uniformity of the architecture.

Section 7 defined the company hub vision, positioning the dashboard as a full enterprise platform with Store, Checkout, ML Access, Device Access, Consulting Access, Government portal, and Documentation viewer.

Section 8 introduced the business-component API model, where each service (ML_V2, DEVICE_API, CONSULTING_API, COURSE_API, etc.) becomes a modular commercial component.

Section 9 described the security model, including token validation rules, expiration, hashing, and API gateway enforcement.

Section 10 consolidated the architecture into a single diagram showing the full flow from client purchase to API response.

Section 11 added the company-level architecture overview, explaining how all layers — frontend, payment, backend, database, and commercial APIs — form a unified enterprise system.

Section 12 provided per-service diagrams for ML models, device APIs, consulting, courses, and marketplace products, illustrating how each service follows the same activation and token-based access pattern.

Section 13 delivered the database schema diagrams, including Users, Purchases, ServiceToken, APIUsage, and Services tables, along with relationship diagrams that show how the database supports the entire commercial ecosystem.

Together, these sections form a complete enterprise architecture document describing how Piezo-Pico to Femtotechnology Sensors Inc. delivers secure, scalable, token-based commercial APIs across all business units. This unified design enables rapid expansion, modular service deployment, and seamless integration of new ML models, devices, consulting packages, and digital products — all protected by a consistent activation and token validation system.